

Change Data Capture Grundlagen (AUTO CDC, SCD Type 1)

Section 3 · Data Transformation and Modelling · Quelle: Kurs 3, Kapitel 11–12, 14

Quellsysteme wie Datenbanken oder SaaS-Anwendungen ändern sich fortlaufend: Neue Kunden werden angelegt, Adressen aktualisiert, Konten gelöscht. **Change Data Capture (CDC)** ist die Technik, mit der solche Änderungen nachverfolgt und in eine Zieltabelle im Lakehouse übertragen werden. Dieses Dokument erklärt die Grundidee von CDC, den Unterschied zwischen SCD Type 1 und Type 2, und wie sich CDC mit der deklarativen `AUTO CDC INTO` -Syntax umsetzen lässt.

1. Was ist Change Data Capture?

CDC bezeichnet die Technik, Änderungen (Inserts, Updates, Deletes) in einer Datenquelle zu erfassen und automatisiert auf eine Zieltabelle anzuwenden, um diese kontinuierlich mit dem aktuellen Stand der Quelle synchron zu halten. Eng damit verbunden ist das Konzept der **Slowly Changing Dimensions (SCD)**, das festlegt, wie historische Änderungen in der Zieltabelle behandelt werden. Zwei Varianten sind besonders verbreitet:

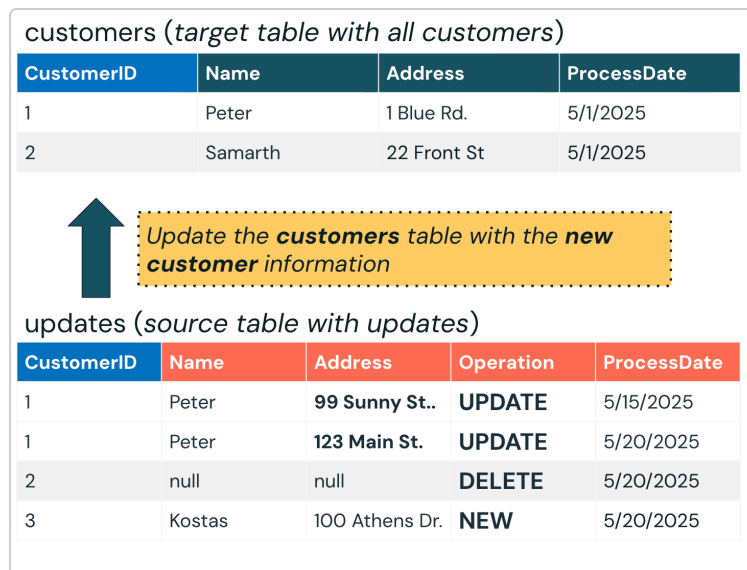
- **SCD Type 1** – überschreibt vorhandene Daten, es wird keine Historie geführt.
- **SCD Type 2** – verfolgt historische Änderungen, indem frühere Versionen eines Datensatzes zusätzlich gespeichert werden.

2. SCD Type 1 – einfach erklärt

Bei SCD Type 1 wird die Zieltabelle stets mit den aktuellsten Werten überschrieben – es gibt keine Möglichkeit, frühere Zustände eines Datensatzes nachzuvollziehen. Ein Beispiel anhand einer Kundentabelle mit drei eingehenden Änderungen:

- **Peter** (Kunde 1): Seine Adresse wird auf den neuesten Stand aktualisiert (basierend auf dem Sequenzfeld `ProcessDate`).
- **Samarth** (Kunde 2): Sein Konto wurde gelöscht → seine Zeile wird aus der Zieltabelle entfernt.
- **Kostas** (Kunde 3): Ein neuer Kunde → sein Datensatz wird eingefügt.

Das Ergebnis ist ein aktueller Schnappschuss aller aktiven Kunden – ohne jede Historie. Das macht SCD Type 1 zur einfachsten CDC-Strategie und ideal für Anwendungsfälle, bei denen ausschließlich der aktuelle, korrekte Datenstand zählt.



SCD Type 1: Updates überschreiben bestehende Zeilen, Deletes entfernen sie, neue Kunden werden eingefügt – ohne Historie.

3. SCD Type 2 – kurzer Ausblick

Im Gegensatz dazu bewahrt SCD Type 2 jede historische Version eines Datensatzes: Statt eine Zeile zu überschreiben, wird eine neue Version eingefügt und die vorherige über Metadaten spalten (`__START_AT` / `__END_AT`) als abgelaufen markiert. Aktive Zeilen tragen ein leeres `__END_AT` , historische und gelöschte Zeilen ein konkretes Datum. Dieses Muster wird in Kapitel 8 dieser Section vertieft; die vorliegenden Grundlagen fokussieren auf SCD Type 1.

4. AUTO CDC INTO: die deklarative CDC-Syntax

Lakeflow Declarative Pipelines bietet mit `AUTO CDC INTO` eine kompakte, deklarative Syntax für CDC-Verarbeitung. Sie ersetzt aufwendige, manuell geschriebene `MERGE INTO` -Logik durch wenige klar lesbare Klauseln. **Wichtiger Hinweis:** `AUTO CDC INTO` löst die ältere `APPLY CHANGES INTO` -API ab – beide verwenden dieselbe Syntax, `APPLY CHANGES INTO` bleibt aber weiterhin funktionsfähig.

```
-- Leere Ziel-Streaming-Table anlegen
CREATE OR REFRESH STREAMING TABLE scd_type_1_customers_silver;

-- CDC-Verarbeitung mit SCD Type 1 (Standardverhalten)
CREATE FLOW scd_type_1_flow AS
AUTO CDC INTO scd_type_1_customers_silver
FROM STREAM customers_bronze_clean
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'delete'
SEQUENCE BY timestamp
COLUMNS * EXCEPT (operation)
STORED AS SCD TYPE 1;
```

Die einzelnen Klauseln im Überblick:

- **KEYS** – definiert eine oder mehrere Spalten als eindeutigen Schlüssel, über den Datensätze identifiziert werden.
- **APPLY AS DELETE WHEN** – legt fest, unter welcher Bedingung ein eingehender Datensatz als Löschung interpretiert wird.
- **SEQUENCE BY** – bestimmt die Spalte, anhand derer verspätet eintreffende Datensätze korrekt in der richtigen Reihenfolge verarbeitet werden.
- **COLUMNS ... EXCEPT (...)** – wählt aus, welche Spalten übernommen werden sollen; hier werden alle außer der reinen Steuerspalte `operation` übernommen.
- **STORED AS SCD TYPE 1** – legt die Historisierungsstrategie fest (Type 1 ist zugleich der Standardwert, wenn diese Klausel weggelassen wird).

`AUTO CDC INTO` übernimmt dabei automatisch mehrere Garantien: inkrementelle/streamende Verarbeitung der CDC-Daten, korrekte Behandlung verspätet eintreffender Datensätze anhand des Sequenzfelds sowie eine einfache Syntax zum Ausschließen von Spalten über `EXCEPT`.

5. Ein Praxisbeispiel mit konkreten Zahlen

In einem Beispiel-Kundendatensatz enthielt die erste Datei 939 neue Kunden, die vollständig als Inserts in die leere Zieltabelle übernommen wurden. Eine zweite, nachgelagerte Datei mit 23 Änderungen enthielt 12 Updates, 1 Delete und 10 neue Kunden. Nach der Verarbeitung zeigte die Zieltabelle exakt $939 + 10 - 1 = 948$ Zeilen – die 12 Updates wurden direkt an Ort und Stelle überschrieben, ohne dass die vorherigen Werte irgendwo erhalten blieben. Das ist das charakteristische Verhalten von SCD Type 1: Nur der aktuelle Zustand zählt.

Ergänzt aus der offiziellen Databricks-Dokumentation: Die vollständige Syntaxreferenz von `AUTO CDC INTO` umfasst neben den hier gezeigten Klauseln weitere optionale Elemente: `IGNORE NULL UPDATES` (NULL-Werte in eingehenden Updates ignorieren statt zu übernehmen), `APPLY AS TRUNCATE WHEN` (nur für SCD Type 1 unterstützt, zum Verarbeiten vollständiger Tabellen-Truncate-Operationen der Quelle) sowie `TRACK HISTORY ON` für granulare Steuerung, welche Spalten bei SCD Type 2 historisiert werden. `STORED AS SCD TYPE 1` ist der Standardwert und muss daher nicht zwingend explizit angegeben werden.

Quelle: [AUTO CDC INTO \(pipelines\) – Databricks-Dokumentation](#)